Severity vs. Priority

Introduction

To be effective in a bug triage negotiation, you need a shared language. Simply saying a bug is "bad" isn't enough. Is it bad because it crashes the system, or because it harms the brand? These are two very different problems. The framework used to create this clarity is the **Severity vs. Priority** model. This reading defines these two concepts, explains who determines them, and shows how to use a matrix to make data-driven decisions about which bugs to tackle first.

What is Severity?

Severity describes the technical impact a bug has on the software system. It is an objective measure of how severely a defect impairs system functionality—answering the question: "How badly does this break the application?"

Severity is determined by Quality Assurance (QA) analysts or developers using a purely technical mindset, evaluating functional impact independently of business deadlines or user popularity (which are handled by Priority).

While naming conventions vary across organizations, Severity is standardly categorized into four levels:

- **Critical:** Complete system failure (for example, application crashes, data loss).
- **High:** A major feature is non-functional, and a core workflow is blocked without a workaround.
- **Medium:** A non-critical feature is malfunctioning, or a core feature has a bug with a reasonable workaround.
- **Low:** A cosmetic issue or minor usability flaw that does not impact core functionality.

What is Priority?

Priority describes the business urgency of fixing a bug—answering the question: "How much does this bug matter to us right now?"

Priority is set by the Product Owner or business leadership, with strong input from QA. It is tied to business goals and influenced by several factors:

Customer Impact: How many users are affected? Are they high-value enterprise customers?

Revenue Impact: Is this bug preventing customers from paying or blocking a key sales demonstration?

Brand Reputation: Does this bug make the company look unprofessional, even if it is minor?

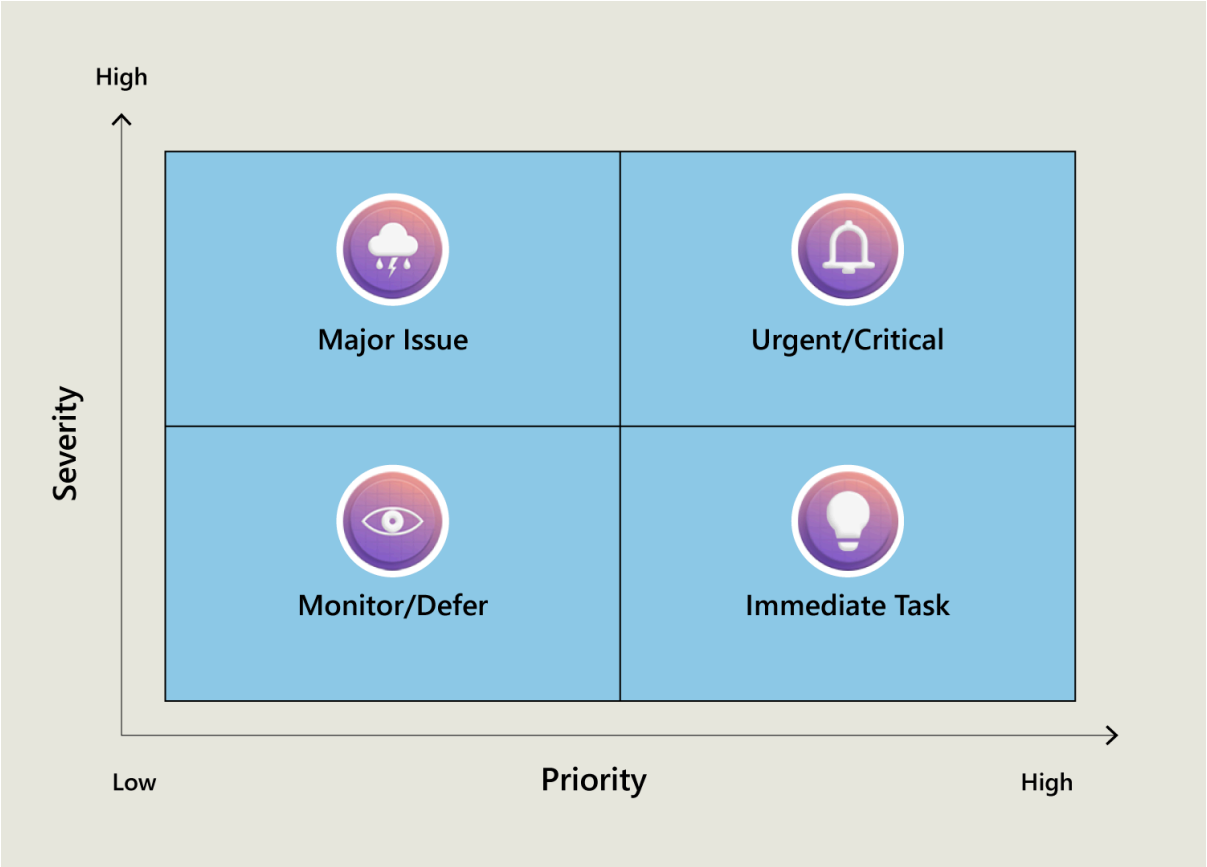
Time Sensitivity: Is a major marketing campaign or industry conference coming up?

Unlike Severity, Priority is dynamic. A low-priority bug can suddenly become high-priority if a major client raises a complaint. The priority scale is straightforward:

- **High:** Fix immediately. Must be addressed in the current or very next release.
- **Medium:** Important, but not an emergency. Schedule for a future planned release.
- **Low:** A "nice-to-have" fix. Place in the backlog to address only when time permits.

The Matrix: Where Severity and Priority Meet

The most challenging bugs are those where Severity and Priority are not the same. This is where the **Severity vs. Priority Matrix** becomes an essential triage tool. It helps visualize that a bug's technical impact does not automatically equal its business urgency.



A 2x2 grid called the Severity vs Priority Matrix The horizontal axis represents Priority and the vertical axis represents Severity The matrix is divided into four distinct quadrants which are High Severity High Priority High Severity Low Priority Low Severity High Priority and Low Severity Low Priority. This grid gives us four distinct categories of bugs, each requiring a different response. High Severity / High Priority: The app crashes during checkout for all users. Fix immediately. Low Severity / High Priority: The company name is misspelled on the homepage. The site works, but it looks unprofessional. Fix quickly. High Severity / Low Priority: A data export feature crashes the app, but only on an obscure legacy browser used by less than 0.1% of users. Defer or do not fix. Low Severity / Low Priority: A typo sits in the third paragraph of the "About Us" page. Place it in the backlog.

This grid gives us four distinct categories of bugs, each requiring a different response.

- **High Severity/High Priority:** The app crashes during checkout for all users. Fix immediately.
- **Low Severity/High Priority:** The company name is misspelled on the homepage. The site works, but it looks unprofessional. Fix quickly.

- **High Severity/Low Priority:** A data export feature crashes the app, but only on an obscure legacy browser used by less than 0.1% of users. Defer or do not fix.
- **Low Severity/Low Priority:** A typo sits in the third paragraph of the "About Us" page. Place it in the backlog.

Conclusion

By separating technical impact (Severity) from business urgency (Priority), you can have a far more productive conversation with your team. Instead of arguing about whether a bug is "bad," you can use the matrix to classify the type of bad it is and make a data-driven decision. This framework is your key to moving from just finding bugs to strategically managing them.